Documentação do Trabalho Prático 2: Desafios Geométricos em Triangulândia

Felipe Almeida Pereira – 2024080051

Novembro 2025

1 Introdução

O problema computacional apresentado consiste em resolver dois desafios de otimização para a prefeitura da cidade fictícia de Triangulândia. Ambos os problemas estão relacionados a formas geométricas, especificamente triângulos, e exigem soluções computacionais eficientes.

O primeiro desafio (Parte 1) é determinar a altura máxima de um muro em formato de triângulo isósceles. Este triângulo deve ser "esculpido"a partir de um conjunto de N pilhas de blocos de madeira já existentes, sem alterar a ordem original das pilhas, aproveitando ao máximo a estrutura existente.

O segundo desafio (Parte 2) é um problema de planejamento urbano. Dentre Z árvores localizadas por coordenadas (X, Y) em um parque, o objetivo é identificar quais três árvores devem ser usadas como vértices para cercar uma nova área de preservação. A escolha deve ser otimizada para que o perímetro da cerca triangular seja o menor possível.

2 Modelagem

Para cada parte do trabalho, foi utilizada uma modelagem distinta para traduzir o problema fictício em estruturas de dados e algoritmos eficientes, conforme as restrições do enunciado (Parte 1 usando um algoritmo guloso/PD e Parte 2 usando divisão e conquista).

2.1 Parte 1: O Muro

O problema das N pilhas de blocos foi modelado como um problema de programação dinâmica em um histograma. As pilhas foram representadas como um vetor (ou lista) de inteiros B, onde $B[i]$ armazena a altura da i-ésima pilha. A forma do triângulo isósceles desejado (conforme a Figura 1 do enunciado) segue o padrão de alturas:

$$[1, 2, \ldots, H-1, H, H-1, \ldots, 2, 1]$$

O algoritmo implementado busca a maior ocorrência dessa forma “piramidal” que pode ser inscrita no histograma B.

2.2 Parte 2: As Árvores

O problema das Z árvores foi modelado como um problema de geometria computacional no plano 2D. As árvores foram representadas como um vetor de uma estrutura `Ponto`, onde cada ponto armazena suas coordenadas (X, Y) e seu índice original (necessário para a formatação da saída). O algoritmo central utilizado foi o de Divisão e Conquista para o "Trio de Pontos Mais Próximos", que é uma adaptação do problema clássico do "Par de Pontos Mais Próximos".

3 Solução

A solução foi dividida nas duas partes, cada uma com seu algoritmo específico.

3.1 Parte 1: O Muro (Programação Dinâmica)

A solução para a Parte 1 encontra a altura máxima do triângulo isósceles H que pode ser centrado em cada pilha i do muro. A resposta final é o valor máximo encontrado. A ideia é que um triângulo de altura H centrado em i é composto por uma rampa ascendente à sua esquerda e uma rampa descendente à sua direita.

A implementação utiliza duas passagens de Programação Dinâmica sobre o vetor de pilhas B:

1) **Iteração para a Esquerda:** É criado um vetor auxiliar L. $L[i]$ armazena a altura da maior rampa ascendente $[1, 2, \ldots, h]$ que pode terminar na pilha i. Esta passagem é calculada da esquerda para a direita com a recorrência:

$$L[i] = \min(B[i], L[i-1] + 1)$$

com o caso base $L[0] = 1$.

2) **Iteração para a Direita:** É criado um vetor auxiliar R. $R[i]$ armazena a altura da maior rampa descendente $[h, \ldots, 2, 1]$ que pode começar na pilha i. Esta passagem é calculada de trás para frente:

$$R[i] = \min(B[i], R[i+1] + 1)$$

com o caso base $R[N-1] = 1$.

3) **Combinação:** O maior triângulo que pode ser centrado na pilha i tem altura:

$$H_i = \min(L[i], R[i])$$

O algoritmo itera por todas as pilhas e encontra a resposta final $H = \max(H_i)$ para todo $i \in [0, N-1]$.

Apesar de ter estrutura semelhante à Programação Dinâmica, o algoritmo é considerado guloso porque, em cada posição, ele faz a escolha local ótima de altura possível sem reavaliações futuras. Em outras palavras, ao calcular $L[i] = min(B[i], L[i-1] + 1)$, o algoritmo decide localmente a maior altura possível para a rampa à esquerda que ainda mantém a forma triangular válida. O mesmo raciocínio é aplicado a $R[i]$.

Essas decisões locais conduzem diretamente à solução global ótima, pois o problema é monotônico , aumentar uma pilha nunca prejudica as escolhas anteriores. Assim, o método é ao mesmo tempo guloso e linear, garantindo a melhor altura possível sem backtracking ou revisões.

3.2 Parte 2: As Árvores (Divisão e Conquista)

A solução para a Parte 2 encontra a tripla de pontos com o menor perímetro através de um algoritmo recursivo de divisão e conquista:

1) **Pré-processamento:** O vetor de Z pontos é ordenado pela coordenada X usando `std::sort`.

2) **Divisão:** O conjunto de pontos P_x (ordenado por X) é dividido ao meio em dois subconjuntos, *Esquerda* e *Direita*, por uma linha vertical.

3) **Conquista:** O algoritmo é chamado recursivamente para ambos os lados, retornando δ_L e δ_R. A melhor tripla encontrada até agora é $\delta = \min(\delta_L, \delta_R)$.

4) **Combinação:** Uma "faixa" (*strip*) é criada contendo todos os pontos cuja distância horizontal à linha central é menor que $\delta/2$. Essa faixa é ordenada por Y e verificada de forma otimizada: se a distância vertical entre dois pontos excede δ, o loop interno é interrompido. O algoritmo mantém a melhor tripla encontrada, respeitando as regras de desempate lexicográfico exigidas.

4 Análise de Complexidade

Sejam N o número de pilhas (Parte 1) e Z o número de árvores (Parte 2).

4.1 Parte 1: O Muro

- **Tempo:** A solução realiza três passagens pelo vetor de pilhas: cálculo de L, cálculo de R e combinação. Cada passagem é $O(N)$, totalizando $O(N)$.

- **Memória:** São usados dois vetores auxiliares (L e R), ambos de tamanho N. Logo, a complexidade espacial é $O(N)$.

4.2 Parte 2: As Árvores

- **Tempo:** A ordenação inicial por X é $O(Z \log Z)$. A etapa de combinação utiliza a relação de recorrência:

$$T(Z) = 2T(Z/2) + O(Z \log Z)$$

  o que resulta em uma complexidade total de $O(Z \log^2 Z)$. Implementações otimizadas podem atingir $O(Z \log Z)$.

- **Memória:** A recursão possui profundidade $O(\log Z)$ e cria vetores auxiliares de tamanho $O(Z)$ em cada nível. A complexidade de memória é, portanto, $O(Z)$.

5 Considerações Finais

A realização deste trabalho prático foi uma experiência de aprendizado extremamente valiosa, pois abordou dois paradigmas de algoritmos fundamentalmente distintos.

A Parte 2 (Divisão e Conquista) foi conceitualmente mais direta, sendo uma adaptação de um problema clássico de geometria computacional. O maior desafio foi implementar a lógica de recursão e a checagem da "faixa" de forma correta, garantindo que a regra de desempate lexicográfico fosse respeitada em todas as comparações.

A Parte 1 (O Muro), surpreendentemente, provou ser a mais desafiadora. A dificuldade não estava na complexidade do código final, mas na interpretação correta do enunciado. O ponto-chave foi perceber que o problema tratava de "esculpir" um triângulo a partir da ordem original das pilhas, e não de reordená-las. Essa percepção levou à solução em Programação Dinâmica eficiente e correta.

6 Referências

- CORMEN, T. H. *et al.* Introduction to Algorithms. 3rd ed. MIT Press, 2009.
- GeeksforGeeks. *Closest Pair of Points using Divide and Conquer.* Disponível em: `https://www.geeksforgeeks.org/closest-pair-of-points-using-divide-and-conquer-al` Acesso em: 05 nov. 2025.
- GeeksforGeeks. *Largest pyramidal shape in a histogram.* Disponível em: `https://www.geeksforgeeks.org/largest-pyramidal-shape-in-a-histogram/`. Acesso em: 07 nov. 2025.
- cppreference.com. *Documentação para std::vector, std::sort, std::stringstream, std::iomanip.* Disponível em: `https://en.cppreference.com/`. Acesso em: 05 nov. 2025.